Ex: 7
Date: 6/8/26

FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

AIM:.

To write a program to implement flow control at the Data Link layer using the sliding window protocol, and to simulate the flow of frames from one node to another.

ALGORITHM:.

1) Start the program.
2) Read the total number of frames to be sent (N), the sender's window size (W), and optionally a frame number to simulate as lost during transmission.
3) Initialize base = 0 (sequence number of the oldest unacknowledged frame) and next seq = 0 (sequence number of the next frame to be sent).
4) Repeat while base < N:
   a) Determine the current sender window as the range [base, base + N - 1].
   b) Send all frames with sequence numbers from next seq up to (base + W - 1), incrementing next seq after each frame, since the window restricts the sender to at most W outstanding (unacknowledged) frames at any time.
   c) If the lost frame lies within the frames just sent:.
      i) The receiver discards the lost frame and any subsequent out-of-order frames.
      ii) The sender times out and retransmits all frames starting from the lost frame (Go-Back-N).
   d) Else (no loss in this batch):

Ex: 7
Date:
6/8/26

```
Enter total number of frames to send : 8
Enter sender's window size : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss): 5

-----------------------------------------
Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
    (Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 5.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

-----------------------------------------
All 8 frames successfully transmitted from Node A to Node B.
```

i) the receiver accepts each frame in order and sends a cumulative acknowledgement (ACK) for it.
ii) on receiving the ACK for the last frame sent, the sender slides its window forward: base = next_seq.

5) Repeat step 4 until all N frames have been transmitted and acknowledged.

6) Stop the program.

PROGRAM:.

```
def simulate (total_frames, window_size, lost_frame):
    base = 0
    next_seq = 0

    While base < total_frames:
window_end = min (base + window_size - 1, total_frames - 1)
print("\n--------")
print(f"Sender window: [Frame {base} - Frame {window_
                                        end}]")

    while next_seq < base + window_size and next_seq
    < total_frames:
print(f"Node A --> Sending Frame {next_seq}")
if next_seq == lost_frame:
    print(f". (Frame {next_seq} lost in transit!)")

    next_seq += 1

if lost_frame is not None and base <= lost_frame
                                    < next_seq:
```

```
print(f"Node B --> Frame {lost_frame} missing.
  Discarding out-of-order frames after it.")
print(f"Node A --> Time out! Resending window starting
  from frame {lost_frame}.")
next_seq = lost_frame
lost_frame = None # loss already handled, retransmission
succeeds continue.

for i in range(base, next_seq):
  print(f"Node B --> Received Frame {i}, Sending
  ACK {i}")
  print(f"Node A --> ACK {next_seq - 1} received.
    Sliding window forword.")
  base = next_seq
  print("\n-----")
  print(f"All {total_frames} frames successfully
  transmitted from NodeA and NodeB.")
  def main():
    total_frames = int(input("Enter tot no-of frames
      to send : "))
    window_size = int(input("Enter Sender's
      window size "))
    lost_frame = int(input(f"Enter frame number to simulate
      as LOST\n(0 to {total_frame - 1}
      enter -1 for no loss) :"))
    if lost_frame == -1:
      lost_frame = None
```

OUTPUT:

Enter total number of frames to send : 8

Enter Sender's window size : 4

Enter frame number to simulate as LOST : 5

(0 to 7, enter -1 for no loss)

Sender window : [Frame 0 - Frame 3]

Node A -> Sending frame 0

Node A -> Sending frame 1

Node A -> Sending frame 2

```
sp mulate (total_frames, window_size, lost_frame)
if __name__ == '__main':
    main()
```

RESULT:

The program to implement flow control at the data link layer using the sliding window protocol was successfully written in python, executed and verified by simulating the flow of frames between two

Ex:8
Date:-

b) NMAP to Discover list host using Nmap scans (ARP, ICMP, TCP) on the Try Hackme platform

AIM:- To study the working of Nmap tool and discover live hosts in a network using different host discovery techniques such as ARP scanning, ICMP, TCP and UDP scanning on Try Hack Me platform.

STEPS:-

1) Open the try hack me platform & select the Nmap room.
2) Study the network topology and identify the source and destination hosts.
3) use ARP requested to discover the MAC Address for local network.
4) observe how ARP packets are broadcast with the network and identify the devices then receives the request.
5) use the ICMP protocol to send ping request.
6) Analyse the packet flow and identify the Nmap exchange of ARP packets.
7) perform ARP host discovery using the Nmap command & identify all active hosts in the local networks.
8) Perform ICMP Echo Scanning and observe the host that respond to the scan.
9) perform ICMP Timestap and analyse the response generated by target system.
10) Perform ICMP Address Mask scanning & result with the precious scans.

Ex:8
Date:

ARP :-

Sudo nmap -PR -sn 10.10.210.0124

Starting Nmap

Nmap scan report for 10.10.210.1

Host is up

Nmap done.

ICMP :-

nmap -PE -sn 10.10.210.0124

Starting Nmap

Nmap scan report for 10.10.210.6

Host is up

Nmap done.

TCP :-

nmap -PS 23 -sn 10.10.210.0124

Sudo nmap -DA80 -sn 10.10.210.0124

11) Perform TCP scanning to identify live hosts by analysis SYN and ACK packets.

12) perform TCP ACK scanning to verify the accessibility of the target hosts.

13) perform TCP ACK scanning to verify how Nmap identify active hosts through UDP packets.

14) Perform a reverse DNS lookups to obtain the hostname of the discovered system.

15) Compare the result obtained from ARP, ICMP, TCP and UDP scanning.

RESULT:

Thus, live hosts in the network were successfully discovered using ARP, ICMP, UDP Scanning techniques with the Nmap tool.